

Trading bot

Author: Yaroslav Riabikov

Goals

Goal of this project is to build end-to-end automated trading bot with web analytics.

Main points:

1. Web crawler continuously scrapes news articles from <https://techcrunch.com/>.
2. Google Gemini API processes text and outputs instructions (Buy/Sell).
3. Bot processes instructions, edits portfolio, saves it and history of operations to database.
4. Webpage shows statistics collected throughout bot's run.

Data

Data to make predictions was scraped from <https://techcrunch.com/> website I was using Latest News section. Bot read articles' links and then processed them one by one from oldest to newest. Processing articles includes reading them from webpage and sending to Google Gemini API to get response. To get stocks' prices I used yfinance library.

Database contains four tables:

1. bot_variables – budget, last_link (link of latest article that was processed)
2. stocks – stocks portfolio (name, count)
3. operations_history – all information about each operation (stock, buy/sell, count, current stock price, remaining budget, time)
4. portfolio_history – stores total portfolio wealth through the time

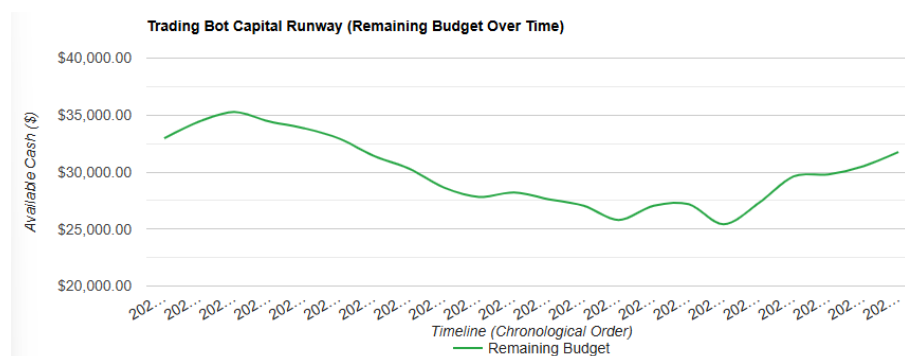
First two tables are used to store portfolio information, last two tables are used to provide statistics, evaluate bot's performance and track its behavior.

Method description

1. Crawler scrapes articles from techcrunch.com, sends to NewsAnalyser
2. NewsAnalyser sends data to Google Gemini API, gets response and returns output to Crawler
3. Crawler gets output from NewsAnalyser and sends to PortfolioOperator
4. PortfolioOperator manages stocks, budget as NewsAnalyser advised

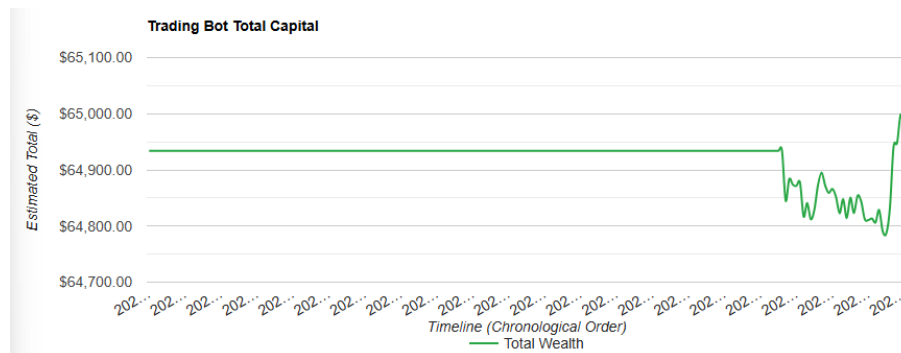
Analysis

First chart is chart representing amount of money available to bot: x-axis represents available cash in dollars, y-axis represents timeline.



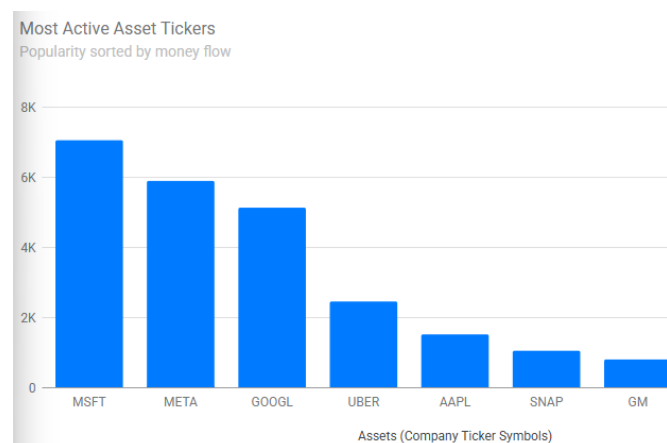
Here we can track if our bot always has enough money and how he spends cash he has. Because of limit per one article, it doesn't spend too much money in single run. We can see that bot doesn't struggle and has enough money.

Second chart is the most important for us: Total capital: x-axis represents estimated total in dollars, y-axis represents timeline.



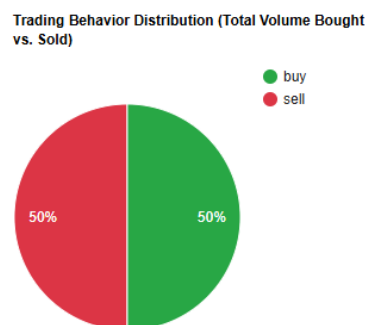
We can see long flat line, it is all runs on weekends when stock market is closed. Then line becomes very messy, but it will have clearer results in long term.

Then we have Barplot that shows us most active tickers: stocks, that bot trades the most: x-axis represents how much money did bot trade in terms of certain stock in dollars, y-axis represents stocks.



Here we can see such large companies as Microsoft, Meta, Google. It is expected, since we get a lot of news about known companies.

Last chart is pie chart: Buy/Sell operation friction:



It is really important for tracking bot's behavior. In my case bot perfectly balances between buying and selling. It means that we won't get such problems as not having money after some time trading or our bot only selling and not buying anything.

As a result, we can see that bot acts quite responsible, it can not spend all money in one go, it spends money gradually, he balances between buying and selling, which means that he will have available cash in long-term, if he will continue having same behavior. He trades mostly well-known stocks.

Conclusion

The most difficult part of the project was to restrict LLM to output meaningless predictions. I needed to apply a lot of normalization so LLM will not break any rules and to ensure that it does not do anything that it was not supposed to do. Another important part was to make architecture clean and upgrade friendly. I decided to create Crawler as main bot and two more ones as supporters so no one will be responsible for too many tasks. However, for more complex systems it will be better for Crawler to be not the main entity, but supporter, since it will be responsible for way too many tasks. The easiest part was probably reading articles from the website. During the project I practiced myself in building architecture from scratch; discovered several interesting libraries; connected crawler, LLM, SQL, flask and JS charts.